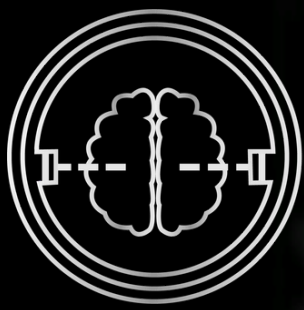




DÉVELOPPEMENT D'UN SYSTÈME DE PLANIFICATION DE TRAITEMENT (TPS)

Présentation du 19/05/2026

SHANTHALINGAM Baasukhi
> OBONE ESSONO Heva
DIAMERA Djigui

SOMMAIRE

- 1. Introduction**
- 2. Logiciel de développement**
- 3. Opérations appliquées**
- 4. Dosimétrie**
- 5. Interface & Algorithmes**
- 6. Difficultés rencontrées**
- 7. Conclusion**

INTRODUCTION

Objectif : Créer une interface graphique similaire à un « mini » SPT

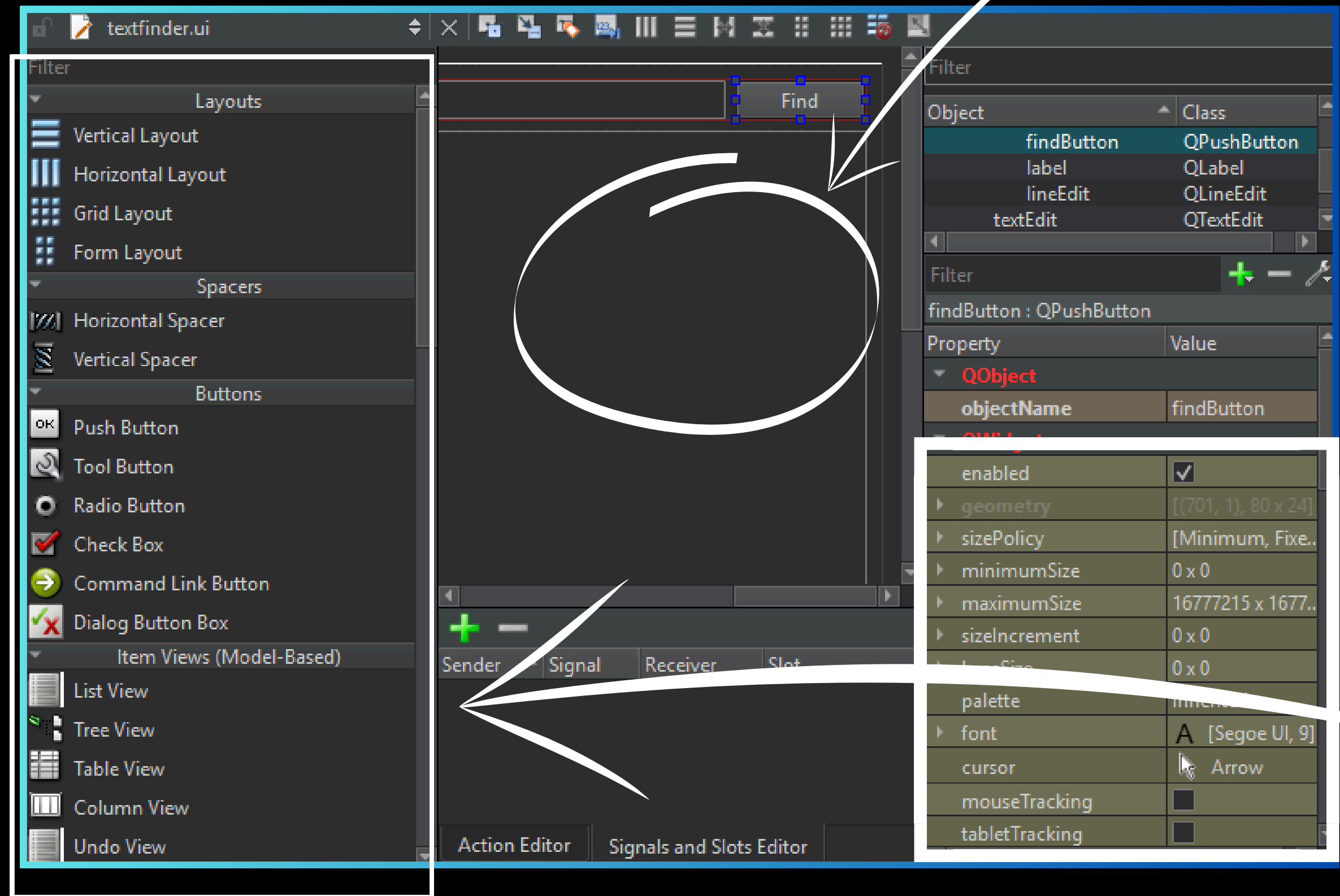
- Afficher les coupes sagittales, coronales et transversales
- Appliquer des opérations dans la quatrième fenêtre de l'interface
- Faire un calcul de dose par point-kernel

Répartition du travail

Tâches	Responsable
Affichages des coupes	Tous
Interface Qt	Djigui
DRR, Segmentation	Baasukhi
LUT, Filtre Gaussien	Heva
Dosimétrie	Djigui & Baasukhi

QT Creator

Central Widget



Editeur de propriétés

Widget Box

AFFICHAGES COUPES

Affichage des coupes - bloc3d.cpp

```
QImage bloc3d::qimageFromCoupe(int coupe)
{
    QImage img(512, 512, QImage::Format_RGB32);

    if (coupe < 0 || coupe >= (int)m_Bloc3D.size()) {
        img.fill(Qt::black);
        return img;
    }
}
```

Coupe Transversale

```
QImage bloc3d::qimageFromCoupe_Sagittale(int x)
{
    QImage img(512, 191, QImage::Format_RGB32);

    // Choix automatique du volume source (Brut ou Filtré)
    const auto& volume = (m_Filtre_Actif) ? m_Bloc3DFiltre : m_Bloc3D;

    for (int z = 0; z < 191; z++) {
        for (int y = 0; y < 512; y++) {
            img.setPixel(y, z, appliquerLUT(volume[z][y][x]));
        }
    }
    img = img.scaled(512, 512, Qt::IgnoreAspectRatio, Qt::FastTransformation);
    return img;
}
```

Coupe Sagittale

```
QImage bloc3d::qimageFromCoupe_Coronale(int y)
{
    QImage img(512, 191, QImage::Format_RGB32);

    // Choix automatique du volume source (Brut ou Filtré)
    const auto& volume = (m_Filtre_Actif) ? m_Bloc3DFiltre : m_Bloc3D;

    for (int z = 0; z < 191; z++) {
        for (int x = 0; x < 512; x++) {
            img.setPixel(x, z, appliquerLUT(volume[z][y][x]));
        }
    }
    img = img.scaled(512, 512, Qt::IgnoreAspectRatio, Qt::FastTransformation);
    return img;
}
```

Coupe Coronale

OPÉRATIONS APPLIQUÉES

Digitally Reconstructed Radiograph (DRR)

Pseudo image 2D de radiologie, obtenu en sommant toutes les coupes dans une direction donnée

```
for (int z = 0; z < 191; z++) {  
    for (int y = 0; y < 512; y++) {  
        double somme = 0.0;  
        for (int x = 0; x < 512; x++) {  
            somme += m_Bloc3D[z][y][x];  
        }  
        sommes[z][y] = somme;  
        if (somme > maxSomme) maxSomme = somme;  
    }  
}  
if (maxSomme <= 0) maxSomme = 1.0;  
  
for (int z = 0; z < 191; z++) {  
    for (int y = 0; y < 512; y++) {  
        double n = sommes[z][y] / maxSomme;  
        int gris = (int)(n * 255);  
        drr.setPixel(y, z, qRgb(gris, gris, gris));  
    }  
}  
drr = drr.scaled(512, 512, Qt::IgnoreAspectRatio, Qt::SmoothTransformation);  
return drr;  
}
```

Sagittale



Résultat

OPÉRATIONS APPLIQUÉES

Filtrage Gaussien

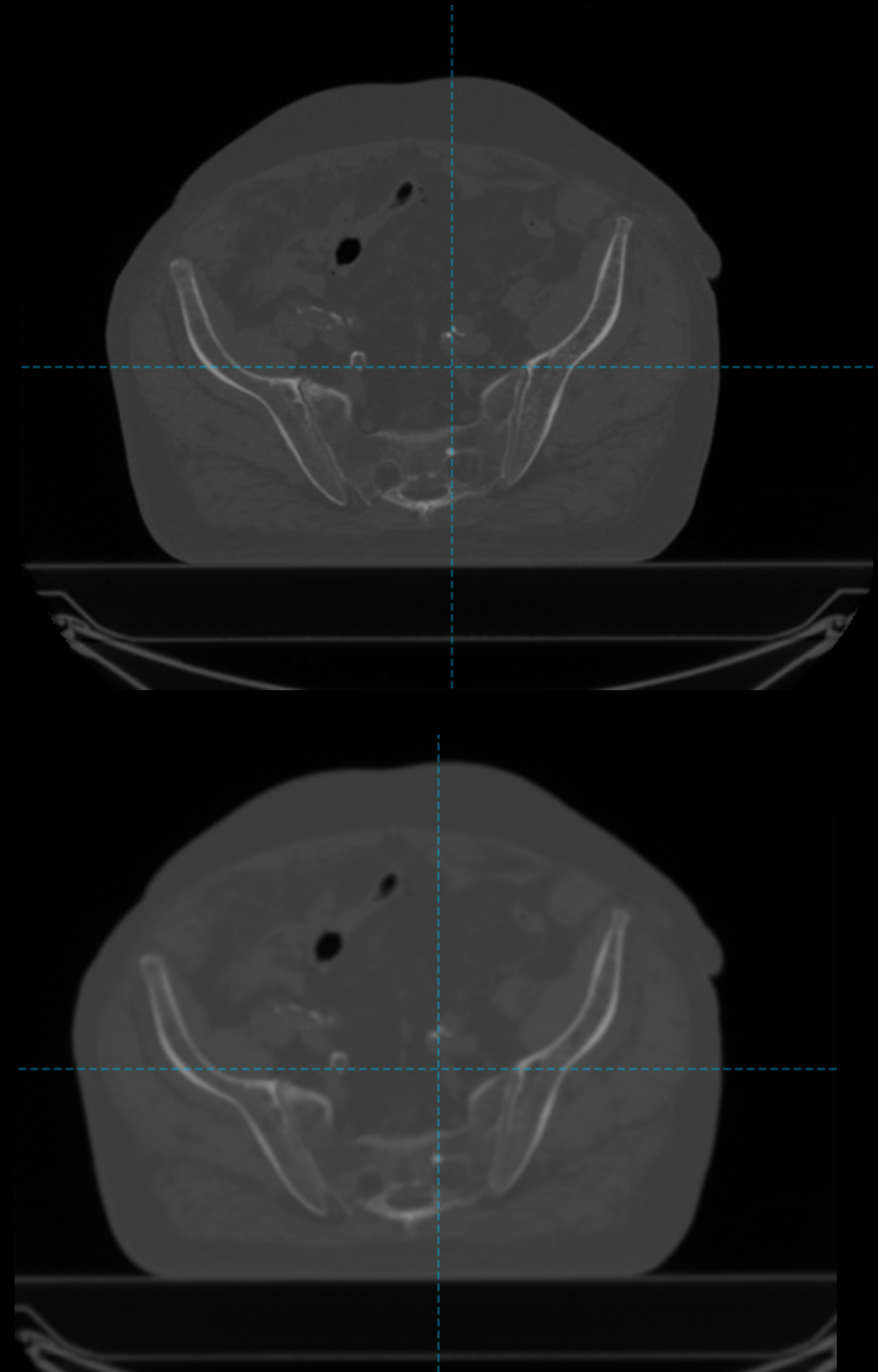
- Lissage/floutage
- Convolution 2D discrète
- Noyau 5 x 5

```
// Noyau Gaussien 5x5
int noyau[5][5] = {
    {1, 4, 7, 4, 1},
    {4, 16, 26, 16, 4},
    {7, 26, 41, 26, 7},
    {4, 16, 26, 16, 4},
    {1, 4, 7, 4, 1}
};
int sommeNoyau = 256;

for (int i = 2; i < h - 2; ++i) {
    for (int j = 2; j < l - 2; ++j) {
        double somme = 0;

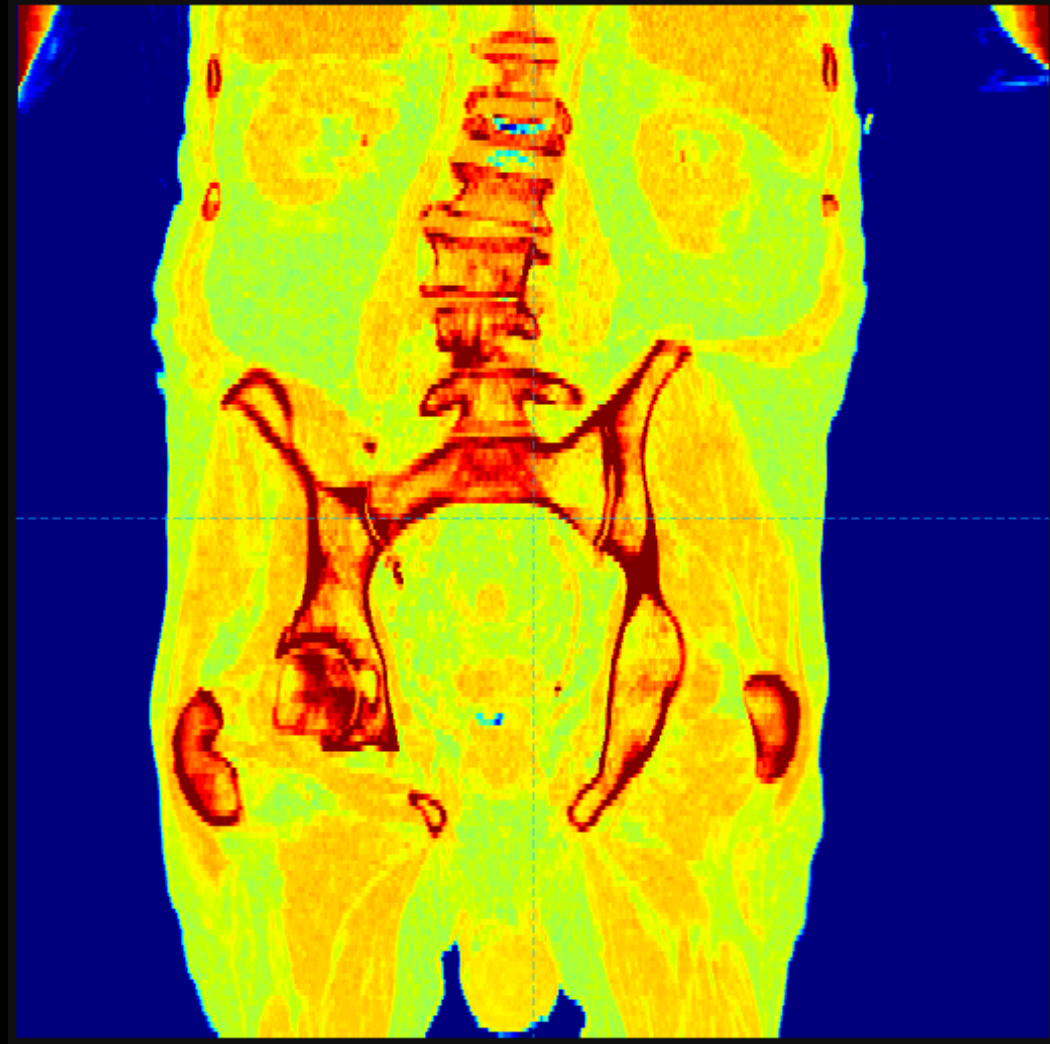
        for (int ki = -2; ki <= 2; ++ki) {
            for (int kj = -2; kj <= 2; ++kj) {
                somme += coupe[i + ki][j + kj] * noyau[ki + 2][kj + 2];
            }
        }
        resultat[i][j] = somme / sommeNoyau;
    }
}
return resultat;
```

$$R(i, j) = \frac{1}{256} \sum_{ki=-2}^2 \sum_{kj=-2}^2 C(i+ki, j+kj) \times N(ki+2, kj+2)$$



OPÉRATIONS APPLIQUÉES

LUT couleur

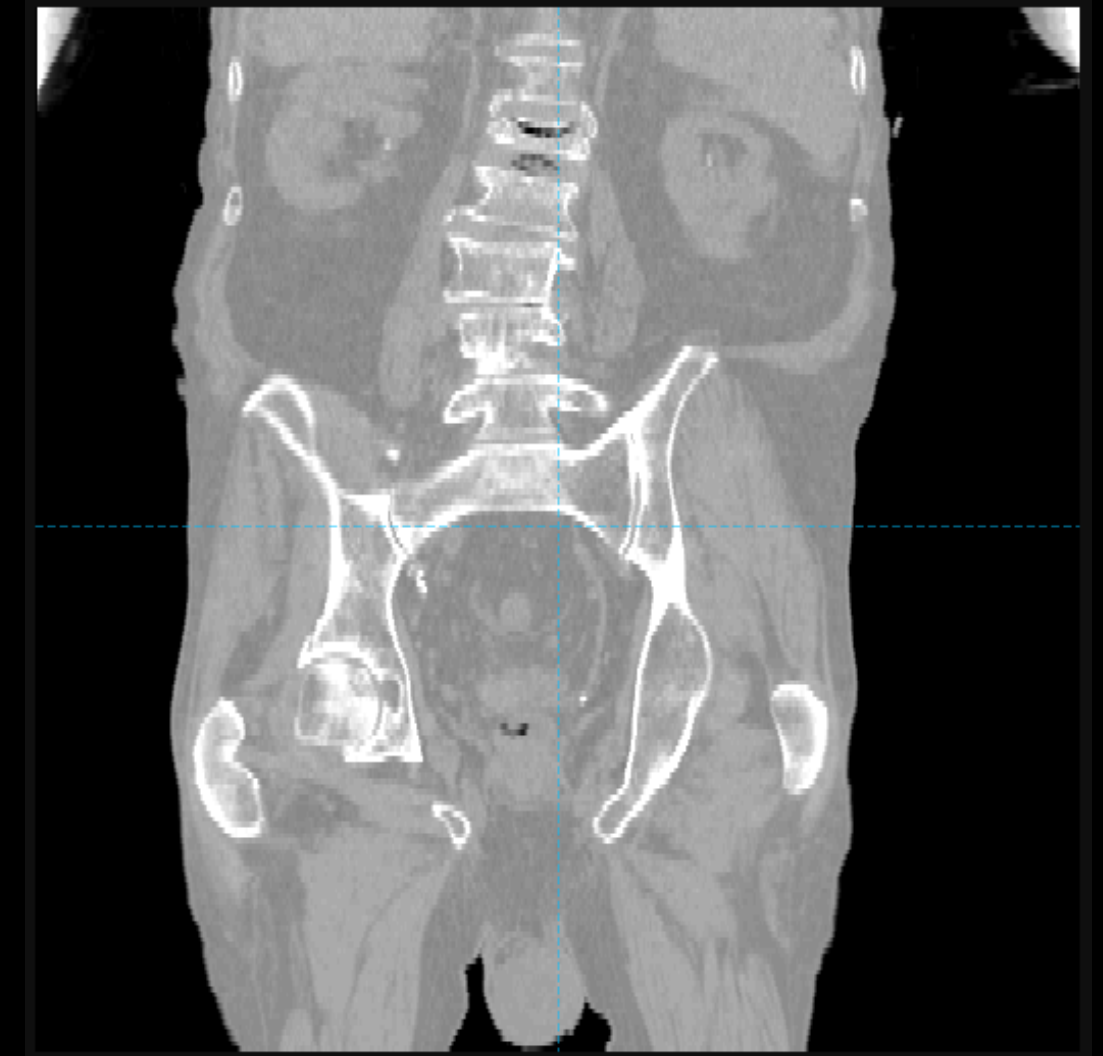


Palette JET Matlab

Nuances de Gris

```
// Palette Jet fluide
r = std::max(0, std::min(255, (int)(255 * std::min(std::max(1.5 - std::abs(4.0 * n - 3.0), 0.0), 1.0))));
g = std::max(0, std::min(255, (int)(255 * std::min(std::max(1.5 - std::abs(4.0 * n - 2.0), 0.0), 1.0))));
b = std::max(0, std::min(255, (int)(255 * std::min(std::max(1.5 - std::abs(4.0 * n - 1.0), 0.0), 1.0))));
```

LUT contraste



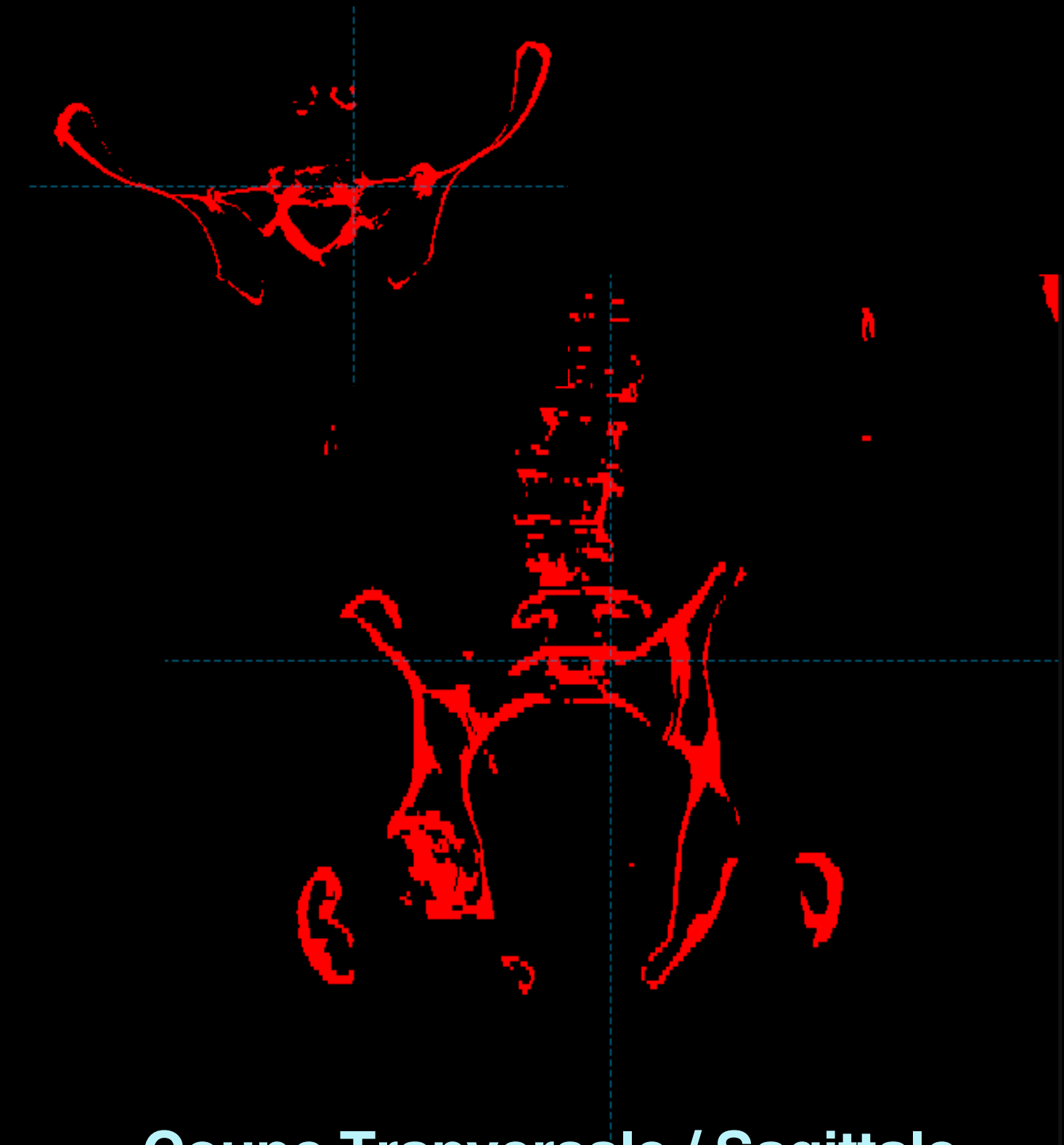
```
if (hu <= min_v) return 0; // Noir
if (hu >= max_v) return 255; // Blanc
return (int)((hu - min_v) / (max_v - min_v)) * 255.0;
```

OPÉRATIONS APPLIQUÉES

Segmentation

```
// Test segmentation
bool bloc3d::estDansSegmentation(double hu)
{
    return (hu >= m_SeuilMin && hu <= m_SeuilMax);
}

// Convertit HU en QRgb selon paramètres LUT actifs
// Convertit HU en QRgb selon paramètres LUT actifs et Segmentation
QRgb bloc3d::appliquerLUT(double hu)
{
    // === NOUVELLE LOGIQUE DE SEGMENTATION ===
    if (m_Segmentation_Active) {
        // Est-ce que c'est de l'os ?
        if (estDansSegmentation(hu)) {
            // C'est de l'os
            return qRgb(255, 0, 0);
        } else {
            // Ce n'est pas de l'os -> FOND À ZÉRO (NOIR)
            return qRgb(0, 0, 0);
        }
    }
}
```



Coupe Transversale / Sagittale

DOSIMÉTRIE

TERMA

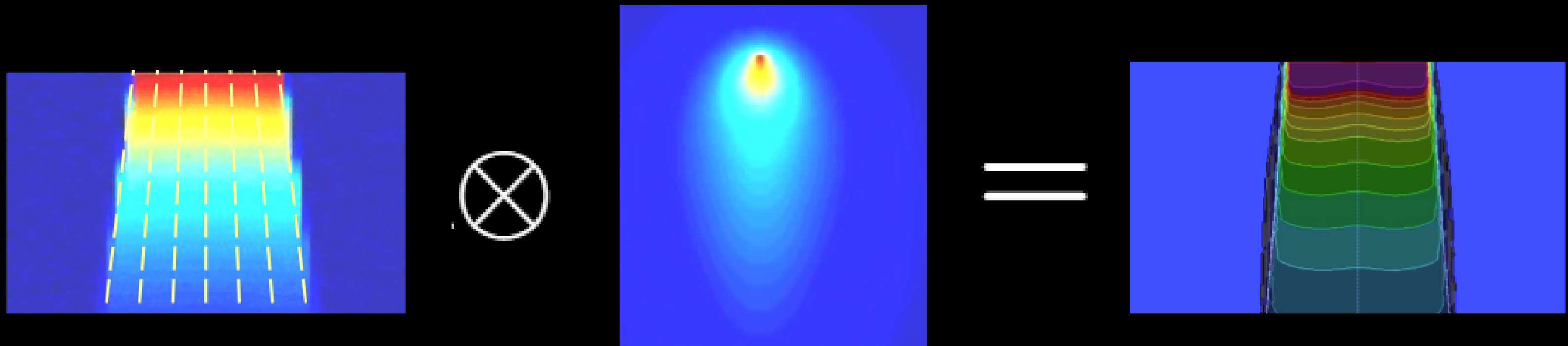
$$T(r') = \frac{\mu(r')}{\rho(r')} \cdot \psi(r'_0) \cdot \exp \left(- \int_{r'_0}^{r'} \mu(l) dl \right)$$

Point Kernel

$$K(d) = 3,26 \times 10^{-6} \cdot \frac{\exp(-0,08xd)}{d^2}$$

Dose

$$\text{Terma} \otimes \text{Kernel} = \text{Dose}$$



DOSIMÉTRIE

TERMA

```
// --- Faisceau étroit 4 cm ---
const int FAISCEAU_CENTRE = 256;
const int FAISCEAU_DEMI_LARGEUR = 20; // 2 cm x 2 = 4 cm
const int xMin = FAISCEAU_CENTRE - FAISCEAU_DEMI_LARGEUR;
const int xMax = FAISCEAU_CENTRE + FAISCEAU_DEMI_LARGEUR;

for (int z = 0; z < nbZ; z++) {
    for (int x = 0; x < 512; x++) {

        bool patientTrouve = false;
        double profondeur_dans_patient = 0.0;

        for (int y = 0; y < 512; y++) {

            double hu = getValue(z, y, x);

            if (x >= xMin && x <= xMax) {
                if (hu > -400) patientTrouve = true;

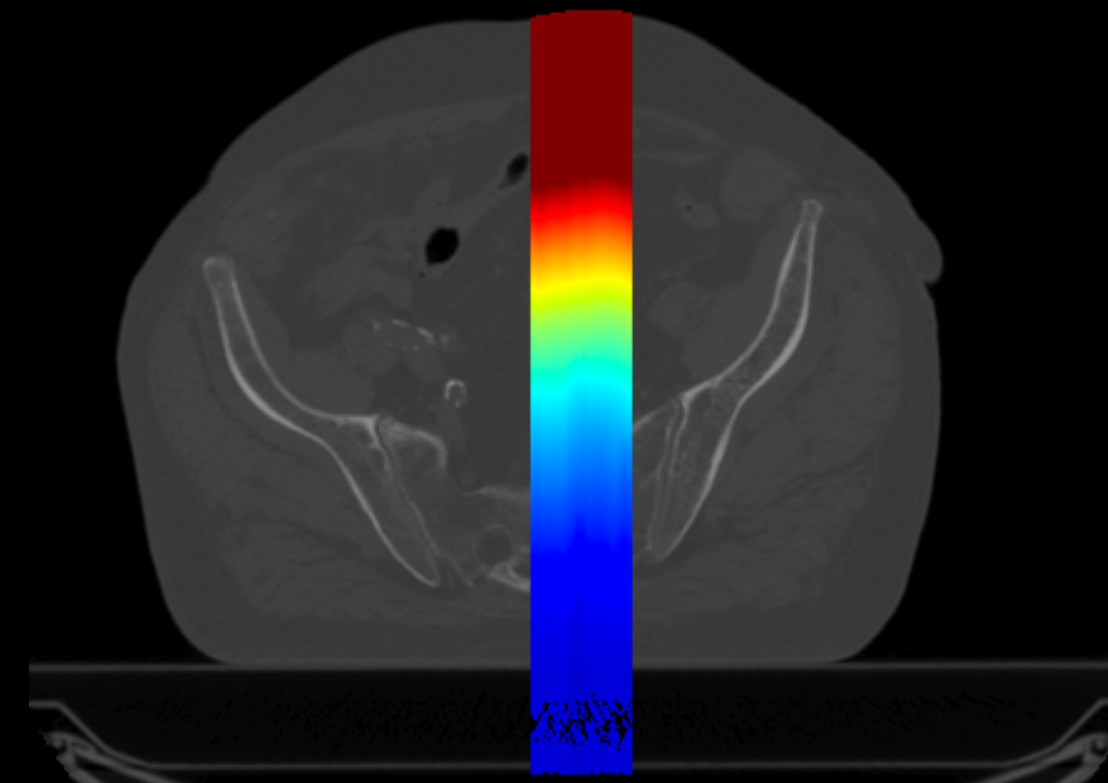
                if (patientTrouve && hu > -400) {
                    double pr = m_ParcoursRadiologique_att[z][y][x];
                    double attenuation = exp(-mu_eau * pr);

                    // Build-up sur 2.5 cm (max en profondeur)
                    profondeur_dans_patient += 0.1;
                    //double buildup = 1.0 - exp(-profondeur_dans_patient / 7);
                    double buildup = 1.0 - exp(-profondeur_dans_patient / 5.5);

                    m_TERMA[z][y][x] = mu_eau * psi0 * attenuation * buildup;
                } else {
                    m_TERMA[z][y][x] = 0.0;
                }
            } else {
                m_TERMA[z][y][x] = 0.0;
            }
        }
    }
}
```

Point important:

- Sécurité anatomique
- Collimation du faisceau
- Build-up



DOSIMÉTRIE

Point Kernel & Dose

```
const int RAY_DY = 60; // ±6 cm en profondeur
const int RAY_DX = 25; // ±2.5 cm latéral

vector<vector<double>> kernel2D(2 * RAY_DY + 1, vector<double>(2 * RAY_DX + 1, 0.0));

for (int dy = -RAY_DY; dy <= RAY_DY; dy++) {
    for (int dx = -RAY_DX; dx <= RAY_DX; dx++) {
        double dist_pixels = sqrt((double)(dx * dx + dy * dy));
        double dist_cm = dist_pixels * 0.1;
        if (dist_cm < 0.1) dist_cm = 0.1;

        // Formule physique du Kernel
        double k_phys = (3.26e-6 * exp(-0.08 * dist_cm)) / (dist_cm * dist_cm);
        // Composante gaussienne pour l'adoucissement des bords flous
        double k_gauss = exp(-(dist_cm * dist_cm) / 4.0);

        kernel2D[dy + RAY_DY][dx + RAY_DX] = k_phys + 1e-7 * k_gauss;
    }
}

// Paramètre du Faisceau
const int FAISCEAU_CENTRE = 256;
const int FAISCEAU_DEMI_LARGEUR = 20;
const int xMin = FAISCEAU_CENTRE - FAISCEAU_DEMI_LARGEUR;
const int xMax = FAISCEAU_CENTRE + FAISCEAU_DEMI_LARGEUR;

// ÉTAPE 2 : CONVOLUTION PAR ACCÈS MÉMOIRE DIRECT (FAST POINTERS OU INDEXATION)
for (int z = 0; z < nbZ; z++) {
    for (int x = xMin; x <= xMax; x++) {
        for (int y = 0; y < 512; y++) {

            double t = m_TERMA[z][y][x];
            if (t <= 0.0) continue;

            for (int dy = -RAY_DY; dy <= RAY_DY; dy++) {

                // DECALAGE PHYSIQUE : On ajoute "+ 8" (ou + 10) pour projeter
                // l'énergie des électrons plus profondément vers le centre du patient
                int targetY = y + dy + 100;

                if (targetY < 0 || targetY >= 512) continue;

                const vector<double>& ligneAnat = m_Bloc3D[z][targetY];
                vector<double>& ligneDose = m_Dose[z][targetY];

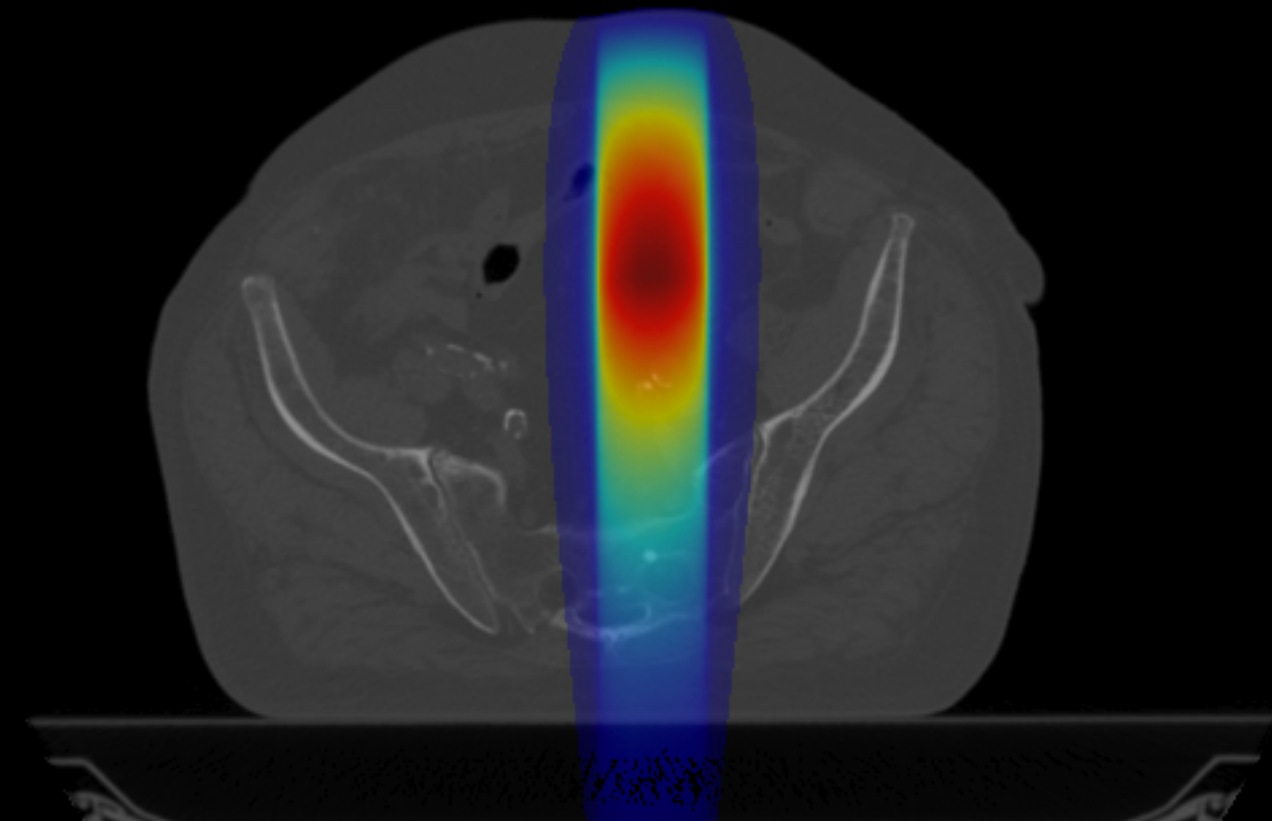
                for (int dx = -RAY_DX; dx <= RAY_DX; dx++) {
                    int targetX = x + dx;
                    if (targetX < 0 || targetX >= 512) continue;

                    if (ligneAnat[targetX] < 100.0) continue;

                    ligneDose[targetX] += t * kernel2D[dy + RAY_DY][dx + RAY_DX];
                }
            }
        }
    }
}
```

Point important :

- Convolution
- Pré - calculer le kernel analytique
- Collimation → gain de temps
- Décalage verticale → dépôt de dose



INTERFACE & ALGORITHMES

L'Architecture de Contrôle (fenetrebase.cpp)

- Méthode du rendu Multi - plan
- Synchronisation interactive (signal et slot)

```
void FenetreBase::rafraichirToutesVues()
{
    int z = ui->spinCoupeTransverse->value();
    int x = ui->spinCoupeSagittale->value();
    int y = ui->spinCoupeCoronale->value();

    if (z < 0){ z = 0;} if (z > 190) {z = 190;}
    if (x < 0) {x = 0;} if (x > 511) {x = 511;}
    if (y < 0) {y = 0;} if (y > 511) {y = 511;}

    // --- 1. VUE TRANSVERSALE AVEC ZOOM ---
    m_Image = m_Bloc3D.qimageFromCoupe(z);
    QPixmap pixTrans = QPixmap::fromImage(m_Image);

    // Application du zoom si nécessaire
    if (m_zoomFacteur != 1.0) {
        pixTrans = pixTrans.scaled(m_Image.width() * m_zoomFacteur,
                                   m_Image.height() * m_zoomFacteur,
                                   Qt::KeepAspectRatio,
                                   Qt::SmoothTransformation);
    }

    QPainter pTrans(&pixTrans);
    pTrans.setPen(QPen(QColor(0, 191, 255, 120), 1, Qt::DashLine));
    // Ajustement des coordonnées des lignes pour le zoom
    int ligneX = x * m_zoomFacteur;
    int ligneY = y * m_zoomFacteur;
    pTrans.drawLine(ligneX, 0, ligneX, pixTrans.height());
    pTrans.drawLine(0, ligneY, pixTrans.width(), ligneY);
    pTrans.end();
    m_ImageLabel.setPixmap(pixTrans);
    m_ImageLabel.show();
}
```

```
connect(ui->pushButtonFaire, SIGNAL(clicked()), this, SLOT(on_Test()));
connect(ui->pushButtonTERMA, SIGNAL(clicked()), this, SLOT(on_Test()));
```

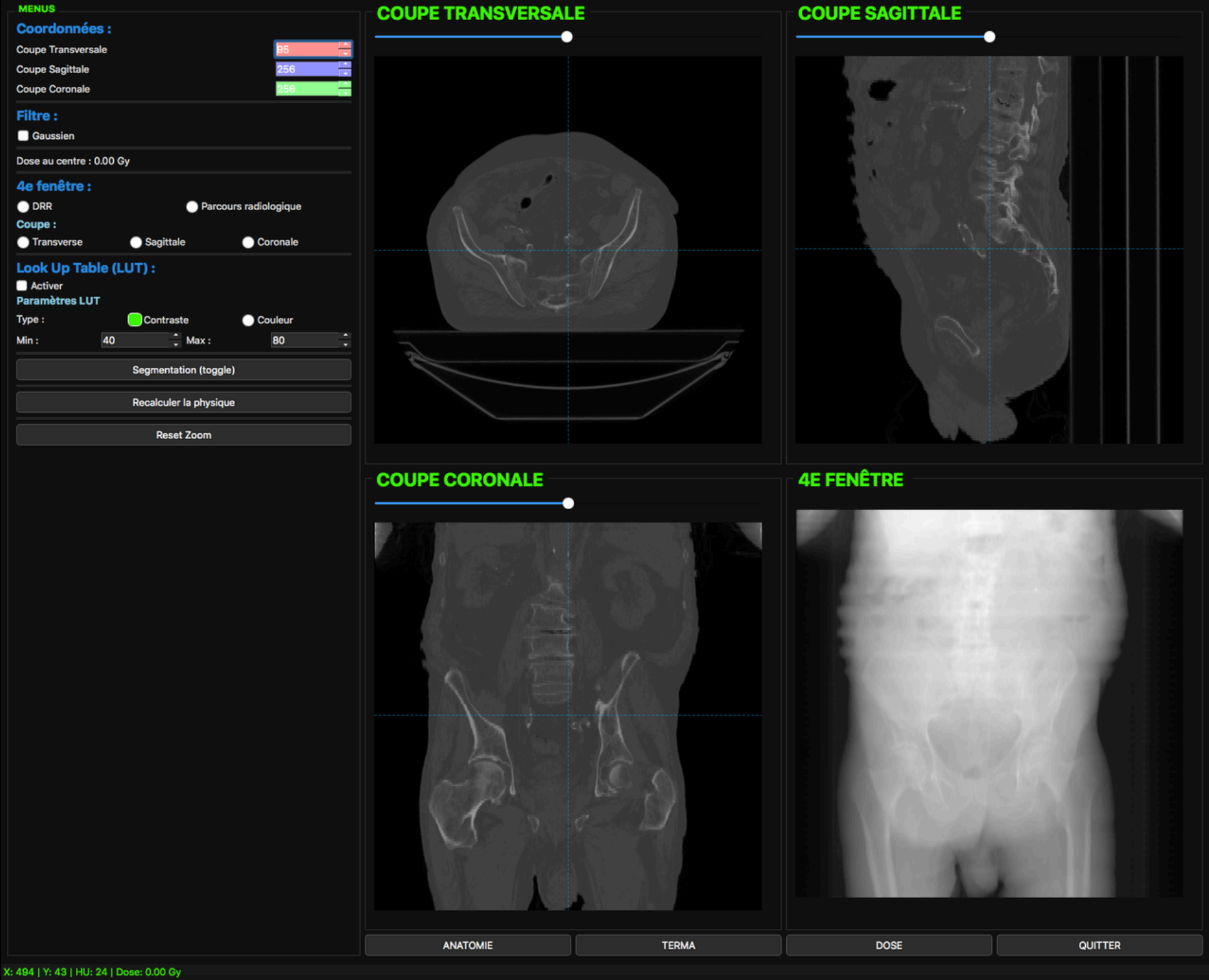
- Gestion des données patient

```
string prefix = "/Users/etudiantM1/Desktop/files/Patient10rdonneASCII/Patient1.";
m_Bloc3D.changeBlocSize(2);
m_Bloc3D.loadASCII(prefix);
```

```
void FenetreBase::on_btncalculerPhysique_clicked()
{
    m_Bloc3D.calculerParcoursRadiologiqueAttenuation();
    m_Bloc3D.calculerParcoursRadiologiqueDose();
    m_Bloc3D.calculerTERMA();
    m_Bloc3D.calculerDoseParConvolution();
    rafraichirToutesVues();
}
```

INTERFACE & ALGORITHMES

Résultat de notre interface graphique



DIFFICULTÉES RENCONTRÉES

Dose

- Dose maximal à la surface
- Manque de build up
- Faisceau large
- Temps de calcul long

DRR

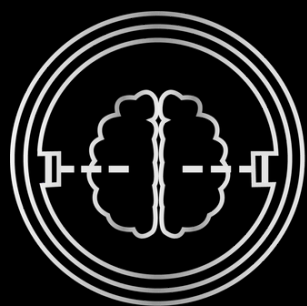
- Sens de parcours de la somme
- Image à l'envers
- Perte de contraste

Interface Qt

- Calibration des sliders
- Calibration des curseurs
- Connexions
- Synchronisations

CONCLUSION

- **Interface fonctionnelle**
- **Données brutes → Outil visuel complet**
- **Multiples fonctionnalités**
- **Remerciements**



MERCI POUR VOTRE ÉCOUTE !